

GRA: Graph-based Reasoning Architecture for LLM Swarm Constructs

bitsoev oleg

June 20, 2026

Abstract

Large Language Models (LLMs) have demonstrated remarkable individual capabilities, yet orchestrating them into collaborative swarms remains an open challenge. We present **GRA** (Graph-based Reasoning Architecture), a framework for constructing and managing LLM swarms through structured, dynamic topologies. In GRA, each LLM agent is a node in a directed acyclic graph; edges encode communication protocols, task dependencies, and reasoning pathways. The swarm construct is defined declaratively via a domain-specific language, allowing automatic generation of execution plans, fault tolerance, and real-time reconfiguration. We evaluate GRA on multi-hop question answering, code synthesis, and debate scenarios, showing that graph-orchestrated swarms outperform static agent ensembles in accuracy (+12%), latency (-18%), and robustness. Our code and benchmarks are available at <https://github.com/qgewq/GRA-LLM-Swarm-Constructs>.

1 Introduction

The rapid evolution of LLMs has enabled single-model systems that can answer questions, generate code, and engage in dialogue. However, complex tasks often require the division of labour, where specialised agents cooperate, debate, or vote to reach a consensus [1, 2]. Existing multi-agent frameworks (AutoGen, CrewAI, MetaGPT) typically rely on linear workflows or rigid hierarchical structures, limiting their adaptability and scalability.

We introduce **GRA**, a Graph-based Reasoning Architecture that treats LLM agents as vertices in a directed acyclic graph (DAG). The graph topology is not a fixed pipeline but a *swarm construct* — a parameterised blueprint that can evolve during execution. Key innovations include:

- A **declarative graph language** to specify agent roles, prompts, and dependencies.
- A **dynamic scheduler** that parallelises independent branches and adjusts the graph on the fly (e.g., spawns sub-swarms).
- **Fault-tolerance mechanisms** such as backtracking and alternative path selection.

The remainder of this paper is structured as follows: Section 2 surveys related work; Section 3 details the GRA architecture; Section 4 describes the swarm construct language; Section 5 presents experimental results; Section 6 concludes and outlines future directions.

2 Related Work

LLM Agents. Tools like LangChain [3], AutoGPT, and BabyAGI have popularised the idea of autonomous LLM agents that plan and execute tasks. They usually follow a single-agent loop with tool access. Our work extends this to multi-agent swarms.

Multi-Agent Collaboration. Recent systems such as AutoGen [4], CrewAI, and MetaGPT organise agents in predefined conversational patterns. Their coordination is largely sequential or relies on centralised message buses. GRA differentiates itself by using an explicit graph structure that can express arbitrary partial orders and can be optimised for parallelism and fault recovery.

Graph-based Reasoning. Graph neural networks and knowledge graphs have been used to enhance LLM reasoning [5]. We focus on the *architectural* use of graphs to coordinate LLM calls, not on embedding-level graph reasoning.

3 GRA Architecture

3.1 Core Abstractions

The GRA framework consists of three layers:

1. **Agent Layer:** LLM instances, each configured with a system prompt, model parameters, and optional tools.
2. **Graph Layer:** A DAG $G = (V, E)$ where $v \in V$ is a node representing an agent invocation, and $e = (u, v)$ indicates that the output of u is fed into the input of v .
3. **Orchestrator:** Responsible for parsing a swarm construct, building the initial graph, scheduling nodes, and handling runtime events (errors, timeouts, task completions).

3.2 Dynamic Scheduling

Nodes are executed as soon as their dependencies are satisfied. Because multiple branches can run concurrently, the orchestrator leverages thread pools or async I/O to maximise throughput. If a node fails (e.g., invalid JSON output from an LLM), the orchestrator can:

- Retry with a refined prompt.
- Activate a fallback node connected via a secondary edge.
- Backtrack and re-plan the sub-graph using a lightweight planner LLM.

3.3 Memory and Context Management

Each agent node maintains a local context window. To avoid context overflow, GRA employs summarisation gates along long dependency chains, controlled by edge properties (e.g., `max_token_budget`).

4 Swarm Construct Language

We designed a YAML-based declarative language to define a swarm construct without writing Python code. An example construct for a three-agent debate is shown below:

```
name: DebateSwarm
entry: proponent
nodes:
  - id: proponent
    agent: gpt-4
```

```

    prompt: "Argue in favour of the motion: {motion}"
- id: opponent
  agent: claude-3-opus
  prompt: "Argue against the motion: {motion}"
- id: judge
  agent: gpt-4-turbo
  prompt: |
    Given the arguments:
    Pro: {proponent.output}
    Con: {opponent.output}
    Decide the winner and explain.
  deps: [proponent, opponent]
output: judge.output

```

The orchestrator compiles this description into a graph, resolves placeholders, and executes the workflow. Recursive constructs (e.g., “debate until convergence”) are expressed via loop macros that generate acyclic sub-graphs per iteration.

5 Experiments

5.1 Setup

We evaluated GRA on three benchmarks:

- **MHQA** (Multi-Hop Question Answering): 500 questions from HotpotQA requiring information from multiple documents.
- **CodeGen-Swarm**: 200 programming tasks from HumanEval and MBPP, solved by a coder–reviewer–tester swarm.
- **DebateScore**: 100 debate motions judged by a panel of three LLM judges.

Baselines included a single best LLM (GPT-4), a static sequential pipeline, and the AutoGen group-chat pattern. All experiments used the same underlying LLM APIs with temperature 0.7.

5.2 Results

Table 1 summarises the findings. GRA outperforms baselines on all tasks, particularly on multi-hop QA where dynamic branching allows the swarm to explore different evidence paths in parallel.

Table 1: Accuracy / pass@1 (%) on three benchmarks.

Method	MHQA	CodeGen	Debate (win %)
Single LLM (GPT-4)	72.1	78.4	55.0
Static pipeline	76.8	82.1	58.3
AutoGen group	79.4	84.5	61.7
GRA	85.3	88.9	68.2

Latency measurements show that GRA’s parallel execution reduces wall-clock time by 18% compared to AutoGen for complex tasks.

5.3 Ablation

Removing the dynamic scheduler (forcing a topological order) drops MHQA accuracy to 80.1%, demonstrating the importance of runtime adaptation. Disabling fallback nodes reduces robustness under high failure rates (10% node failure leads to 22% drop in success rate).

6 Conclusion

We introduced GRA, a Graph-based Reasoning Architecture that enables the construction of flexible, resilient LLM swarms. By representing collaboration as a DAG and providing a high-level declarative language, GRA simplifies the design of complex multi-agent workflows while improving performance through dynamic scheduling and fault tolerance. Future work will explore self-optimising graphs, integration with external knowledge bases, and formal verification of swarm behaviour.

Acknowledgements

We thank the open-source community for providing the tools that made this project possible.

References

- [1] Y. Li et al., “Collaborative LLM agents,” *NeurIPS Workshop*, 2023.
- [2] C. Wang et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” *arXiv:2308.08155*, 2024.
- [3] LangChain, <https://www.langchain.com>, 2024.
- [4] Microsoft AutoGen, <https://microsoft.github.io/autogen/>, 2024.
- [5] M. Yasunaga et al., “QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering,” *NAACL*, 2021.